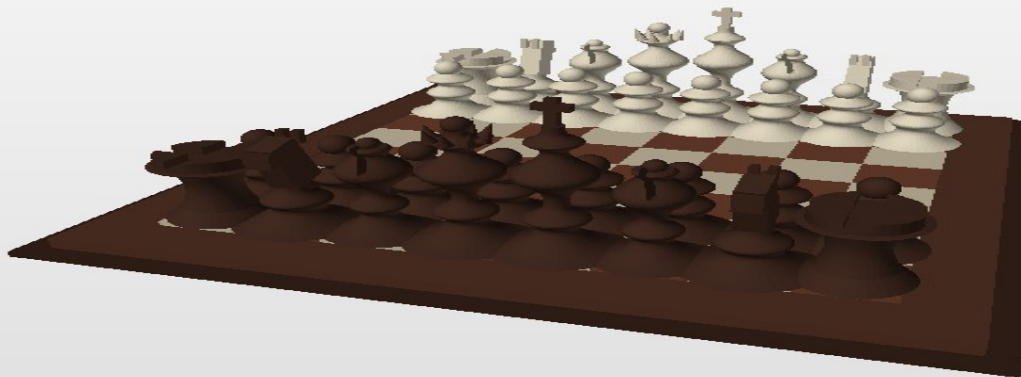


add.py 2.0

Trimačiai modeliai, sukurti vien tik programiniu kodu



Kas naujo 2.0 versijoje

Loginės operacijos

union, intersect, difference – sąjunga, sankirta ir skirtumas. Parašyta nuo nulio, be jokių bibliotekų.

OBJ, PLY ir STL

save("modelis.obj") sukuria .obj ir .mtl – tiesiai į Sketchfab. STL tinka 3D spausdinimui.

Modelio taisymas

clean() sukliauja viršūnes, pašalina pasikartojančias ir vidines sienas; heal() užtaiso plyšelius.

check()

Pasako, kiek turite daugiakampių ir spalvų, ar paviršius uždaras ir ar užduoties reikalavimai jau įvykdyti.

Storis paviršiams

thickness= paverčia ploną parametrinį lakštą tikru kūnu, apšviestu iš abiejų pusių.

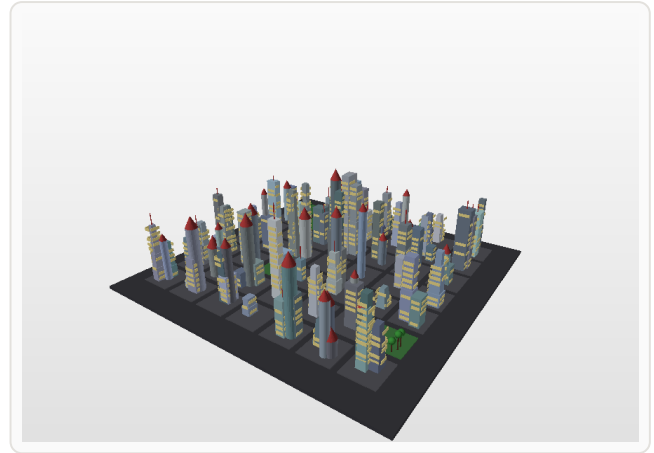
130 funkcijų

Aiškesni vardai (box, tube, frame, revolve, sweep), bet visi seni vardai tebeveikia.

Viskas, kas parašyta senajai 1.2 versijai, veikia be jokių pataisymų.

Kūrybinė užduotis

- Sukurti 3D modelį naudojant tik pirminį programos tekstą.
- Modelio failo formatas – OFF arba OBJ.
- Bent 10 000 daugiakampių.
- Bent 3 skirtingos spalvos.
- Algoritme privalo būti for arba while ciklas.
- Bent 1 parametras, nuo kurio priklauso modelio forma.
- Negalima naudoti 3D modeliavimui skirtų programų.
- Modelį galima kurti individualiai arba komandoje dviese.



Vienas skaičius (SEED) nulemia visą miestą

add.check()

pasako, kurie reikalavimai jau įvykdyti.

Rekomenduojami įrankiai

Python 3

Daugiau nieko diegti nereikia. add.py naudoja tik math ir random.

add.py

Vienas failas šalia jūsų programos. Parsisiunčiamas iš GitHub.

tools/preview.py

Modelio peržiūra be jokios kitos programos: sukuria PNG paveikslėlį.

GeoGebra

Patogu patikrinti kreivių ir paviršių formules prieš rašant kodą.

MeshLab

OFF peržiūra ir konvertavimas. Nuo 2.0 nebebūtinas – OBJ išsaugo pats add.py.

Sketchfab.com

.obj + .mtl viename archyve – ir modelį galima pasukioti naršyklėje.

Modelis – tai du sąrašai

```
OFF
8 6 0
-1 -1 -1
-1 -1 1
...
4 0 4 5 1 255 0 0
4 0 1 3 2 255 0 0
...
```

OFF – formato vardas

8 6 0 – viršūnių, sienų ir briaunų skaičius

toliau – viršūnių koordinatės

4 0 4 5 1 – keturkampis, jungiantis viršūnes 0, 4, 5 ir 1

255 0 0 – sienos spalva

Visos šios bibliotekos braižymo funkcijos daro tą patį: apskaičiuoja taškus ir prideda sienas.

```
import add

add.box([0, 0, 0], 2, "red")
add.save("kubas.off")
```

Tas pats kubas – keturios eilutės.

Vidinė ir išorinė siena

- Žvelgiant į modelį iš išorės, viršūnių indeksus reikia išdėstyti prieš laikrodžio rodyklę.
- Žvelgiant iš vidaus – pagal laikrodžio rodyklę.
- Apversta siena atvaizduojama juoda arba visai nerodoma.
- Bibliotekos figūros orientuojamos pačios – galvoti apie tai reikia tik rašant `add.polygon(...)` ranka.
- **`add.fix_normals(M)` sutvarko visą modelį; `add.check()` pasako, ar paviršius uždaras.**



Tas pats balnas: lakštas, dvipusis ir tūrinis

Scena, sluoksniai ir modeliai

```
add.box([0, 0, 0], 1, "red")
plyta = add.layer()      # scena vėl tuščia

for i in range(10):
    add.mesh(add.move(plyta, [i * 1.2, 0, 0]))

add.save("siena.off")
```

**Sukurk vieną kartą –
padėk daug kartų.**

Transformacijos niekada nekeičia joms
perduoto modelio – jos grąžina naują.

layer()

Paima sceną kaip modelį ir palieka sceną tuščią.

mesh(M)

Grąžina modelį atgal į sceną.

merge([...])

Sujungia kelis modelius į vieną.

push() / pop()

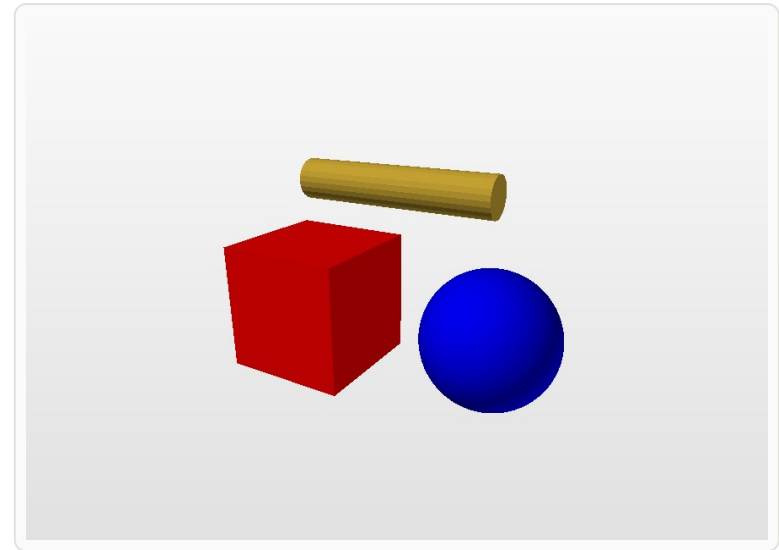
Funkcijoje, kurianti atskirą detalę: padeda sceną į šalį
ir po to grąžina.

Pirmasis modelis

```
import add

add.box([0, 0, 0], 2, "red")
add.sphere([3, 0, 0], 1, 20, "blue")
add.cylinder([0, 2, 0], [3, 2, 0],
            0.3, 24, "gold")

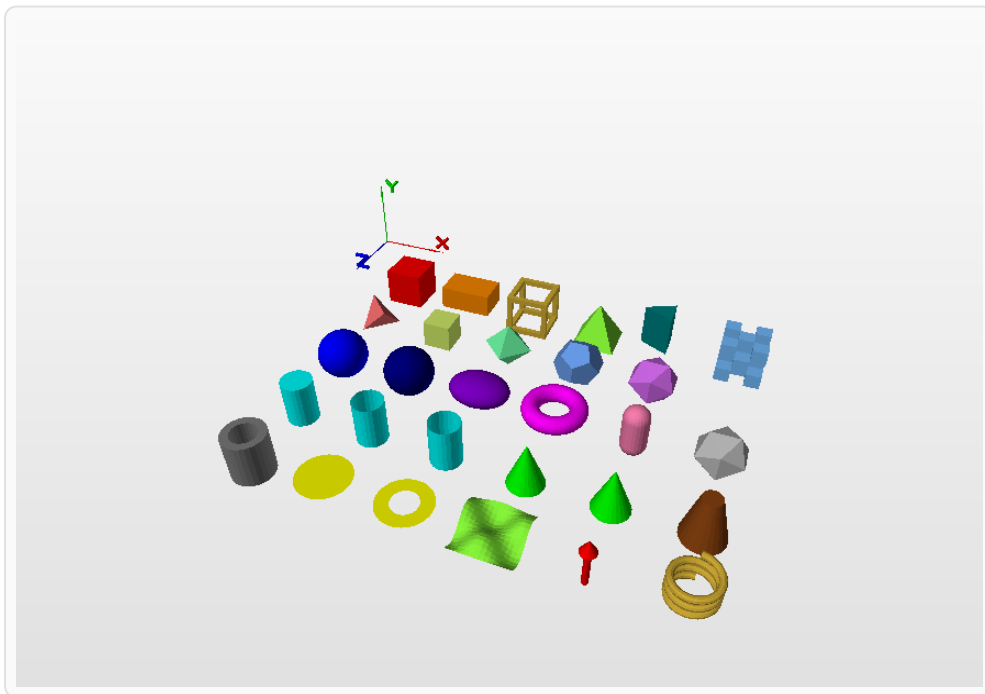
add.check()
add.save("pirmas.off")
```



vertices	2460	
!! polygons	2478	(need 10000)
OK colours	3	(need 3)
OK closed surface	yes	

check() ataskaita: dar trūksta daugiakampių.

Paruoštos figūros



Kūnai

box, cuboid, frame, pyramid, prism, polyhedron, voxels

Apvalūs

sphere, uvsphere, ellipsoid, torus, capsule, cylinder, tube, cup, cone, cone_open, frustum, pipe

Plokšti

polygon, triangle, quad, disc, ring, grid

Pagalbiniai

arrow, helix, axes, glyph

examples/02_primitives.py – visos figūros vienoje programoje.

parametric() – pagrindinė funkcija

```
def toras(u, v):  
    return [(5 + math.cos(u)) * math.cos(v),  
            math.sin(u),  
            (5 + math.cos(u)) * math.sin(v)]  
  
add.parametric(toras, 0, 2*math.pi, 40,  
               0, 2*math.pi, 80,  
               "gold", wrap_u=True, wrap_v=True)
```



wrap_u, wrap_v

Uždaro paviršių be siūlės ir be pasikartojančių viršūnių.

thickness=

Suteikia lakštui tikrą storį – gaunamas uždaras kūnas.

double_sided=

Pigesnis variantas: kiekviena siena ir jos apversta kopija.

Parametriniai paviršiai: viena lygtis, dvylika formų

Gielio superformulė (2003)

$$r(t) = (|\cos(m \cdot t/4)/a|^{\frac{1}{n_2}} + |\sin(m \cdot t/4)/b|^{\frac{1}{n_3}})^{\frac{1}{n_1}}$$

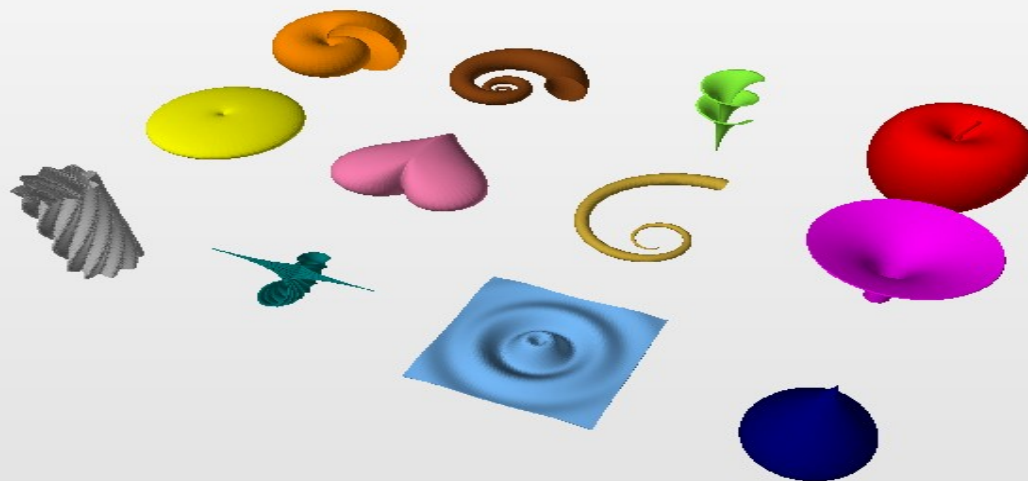
Šeši skaičiai – ir gaunama jūrų žvaigždė, gėlė, kristalas arba paprasčiausias rutulys.

Būtent to ir prašo užduotis: vienas parametras, keičiantis formą.



`examples/08_surfaces_superformula.py`

Paviršiai, kurie panašūs į daiktus



Kriauklė, sraigė, Dini paviršius, obuolys, citrina, širdis, ragas, trimitas, banguota kolona, „breather“, raibuliai, lašas.
`examples/09_surfaces_nature.py`

Grafikai ir egzotiški paviršiai



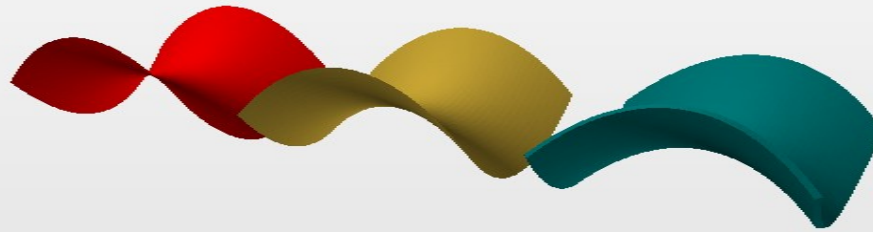
$y = f(x, z)$ · 10_height_fields.py



Miobijus, Kleinas, Boy, Romos paviršius · 07_surfaces_exotic.py

Vienpusiai paviršiai neturi „išorės“. Duokite jiems storį – ir jie tampa įprastais dvipusiais kūnais.

Kai paviršius iš vienos pusės juodas



Paprastas lakštas

Viena gera pusė, viena „negyva“.

`double_sided=True`

Kiekviena siena ir jos apversta kopija. Pigu, storio vis dar nėra.

`thickness=0.1`

Tikras kūnas su sienele: uždaras, spausdinamas, teisingas iš visų pusių.

revolve() – sukinys

```
def vaza(t):  
    return [1 + 0.4*math.sin(3*t), t]  
  
add.revolve(vaza, [0, 0, 0], [0, 1, 0],  
            0, 4, 60, 40, "teal")
```

profilis(t) grąžina [spindulys, aukštis]

caps=True uždaro galus – gaunamas kūnas, tinkamas loginėms operacijoms

angle= mažesnis už pilną apsisukimą išpjauna pleištą



Senasis spin3D(A, B, S, ...) tebeveikia – tai tas pats sukinys be dangtelių. [examples/14_lathe_gallery.py](#)

twist ir scale gali būti ir funkcijos nuo padėties kelyje – forma keičiasi eidama. Dvylika pavyzdžių: [examples/13_sweep_and_twist.py](#)



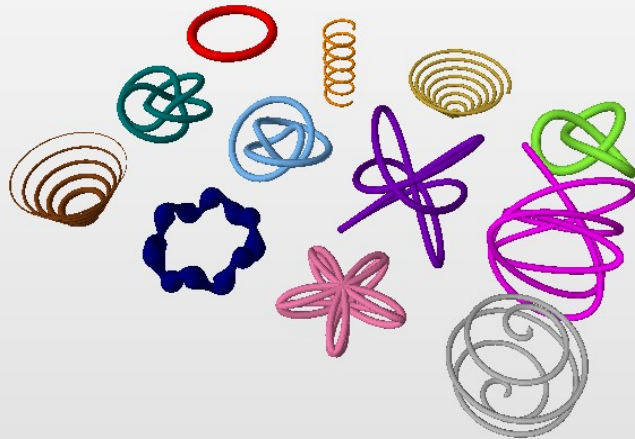
2D forma ištempiama į 3D; pakeliui sukama ir siaurinama.

Skerspjuvis stumiamas išilgai bet kokio 3D kelio.

Paviršius, uždengiantis skerspjuvių seką.

Plokščia juosta, einanti 3D keliu.

curve() – kreivės kaip vamzdžiai



```
def mazgas(t):  
    return [math.sin(t) +  
            2*math.sin(2*t),  
            -math.sin(3*t),  
            math.cos(t) -  
            2*math.cos(2*t)]  
  
add.curve(mazgas, 0, 2*math.pi,  
          300,  
          18, 0.28, "lime", True)
```

Spindulys gali būti ir funkcija $r(t)$ – vamzdis storėja ir plonėja. Vamzdis pats nesisuka: išilgai kreivės nešamas minimaliai sukantis rėmas.

Transformacijos

move, place

Pastūmimas; place pastato centrą į nurodytą tašką.

rotateX/Y/Z, rotate

Sukimas apie ašį – taip pat ir apie bet kokią kryptį.

zoom, stretch, fit

Mastelis: vienodas, atskiras kiekvienai ašiai arba pagal dydį.

mirror

Atspindys plokštumoje; sienų kryptis apsukama.

twist, bend, taper

Susukimas, sulenkimas, siaurėjimas.

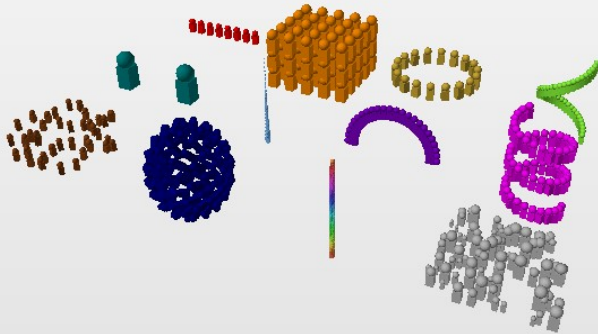
deform

Bet kokia jūsų funkcija $f(p) \rightarrow$ naujas taškas.

Visos jos grąžina naują modelį – originalas nepakinta, todėl jas galima jungti į grandinę:

```
M = add.move(add.rotateY(add.zoom(M, 0.5), 0.4), [0, 3, 0])
```

Viena detalė, daug kopijų



```
array_linear(M, step, n)
```

```
array_grid(M, steps, counts)
```

```
array_radial(M, n, axis, P)
```

```
array_mirror(M, point, normal)
```

```
repeat(M, n, step)
```

```
spirale = add.repeat(plyta, 40,  
    lambda M, i: add.move(  
        add.rotateY(M, i * 0.3),  
        [0, i * 0.2, 0]))
```

Spalvos

```
add.box(c, 1, "red")           # vardu
add.box(c, 1, "#ff8800")      # hex
add.box(c, 1, [255, 136, 0])  # RGB

add.hsv(0.3, 0.7, 1.0)        # atspalvis
add.gradient(t, "navy", "gold") # perėjimas

M = add.color_by(M,
    lambda p: add.hsv(p[1] / 10.0))
```

Užduočiai reikia bent trijų spalvų – bet gražiau, kai jos ką nors reiškia.

color(M, spalva)

Perdažo visą modelį.

color_by(M, fn)

Spalva pagal sienos padėtį – vaivorykštės, aukščio žemėlapiai.

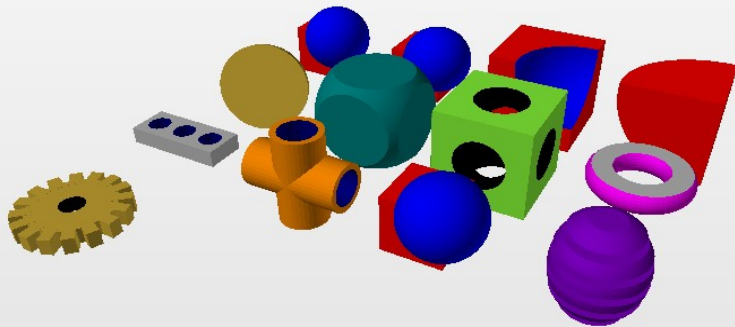
color_gradient

Perėjimas nuo vienos spalvos iki kitos išilgai ašies.

color_random

Kiekvienai sienai – sava atsitiktinė spalva.

Loginės operacijos su kūnais



union(A, B)

Sajunga: viskas, kas yra bet kuriame kūne. Paslėptos vidinės sienos dingsta.

intersect(A, B)

Sankirta: tik tai, kas bendra.

difference(A, B)

Skirtumas: A su iškirptu B. Taip daromos skylės ir įpjovos.

cut(M, taškas, normalė)

Pjūvis plokštuma – pigiau.

```
add.cuboid([0,0,0], [4,1,4], "brown")
plokste = add.layer()
add.cylinder([0,-1,0], [0,1,0], 0.6, 32, "black")
grezlas = add.layer()
add.mesh(add.difference(plokste, grezlas))
```

Kaip veikia loginės operacijos

1. Perpjaunama

Kiekviena siena perpjaunama ten, kur ją galėtų kirsti kito kūno trikampiai. Kaimynai randami per erdvinį tinklėlį, todėl toli esanti siena nepjaustoma.

2. Klausinama

Kiekvienos dalies klausinama: ar ji viduje, išorėje, ar guli ant kito kūno paviršiaus? Atsako spindulys, skaičiuojantis susikirtimus.

3. Atrenkama

Sąjunga palieka tai, kas išorėje; sankirta – kas viduje; skirtumas – A išorę ir apverstą B vidų.

Apie 600 eilučių gryno Python. Jokių bibliotekų.

6000 sienų sfera minus dėžė – apie 0,5 s. 25 000 sienų – apie 2 s.

Loginėms operacijoms reikia uždarytų kūnų. Jei rezultatas keistas – pirmiausia patikrinkite abu pradinius kūnus su `add.check()`.

Modelio taisymas ir tikrinimas

clean(M)

Suklijuoja sutampančias viršūnes, pašalina nulinio ploto ir pasikartojančias sienas, išmeta vidines sieneles ten, kur du kūnai liečiasi.

heal(M)

Užtaiso plyšelius ten, kur briauna praeina pro svetimą viršūnę.

fix_normals(M)

Nukreipia visas sienas ta pačia kryptimi, uždarame modelyje – į išorę.

triangulate(M)

Paverčia visas sienas trikampaiais.

```
modelis = add.clean(add.layer())  
add.mesh(modelis)  
add.check()
```

vertices	136768
OK polygons	163132
OK colours	6
OK closed surface	yes
volume	60.286

Sudėjus du besiliečiančius kubus, kiekvienas pasilieka savo sienelę toje vietoje, kur jos niekas nemato. clean() jas randa ir pašalina.

Modelio išsaugojimas ir viešinimas

```
add.save("modelis.off")    # kurso formatas
add.save("modelis.obj")    # + .mtl spalvoms
add.save("modelis.ply")
add.save("modelis.stl")    # 3D spausdinimui
```

MeshLab konvertavimo žingsnis nebereikalingas – add.py OBJ ir MTL parašo pats.

Sketchfab

Sudėkite .obj ir .mtl į vieną archyvą ir įkelkite – modelį galės pasukioti bet kas naršyklėje.

3D spausdinimas

clean(..., normals=True), tada save("m.stl"). Paviršius turi būti uždaras.

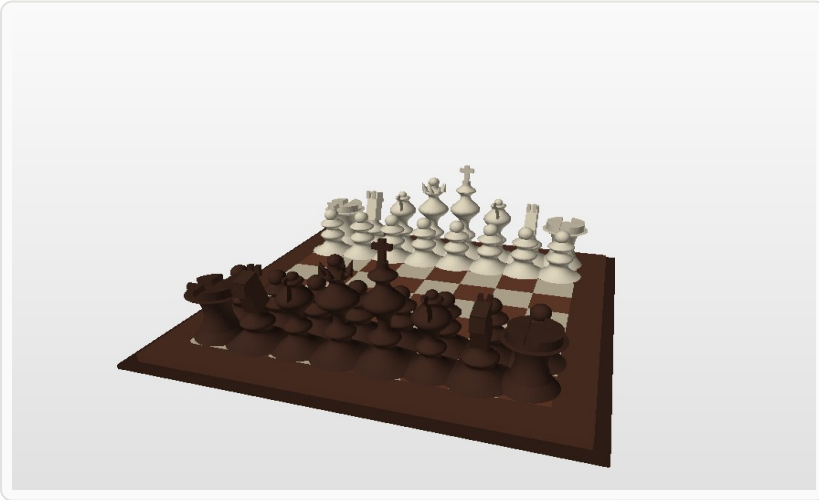
Peržiūra be MeshLab

python3 tools/preview.py modelis.off sukuria PNG.
Irgi be jokių bibliotekų.

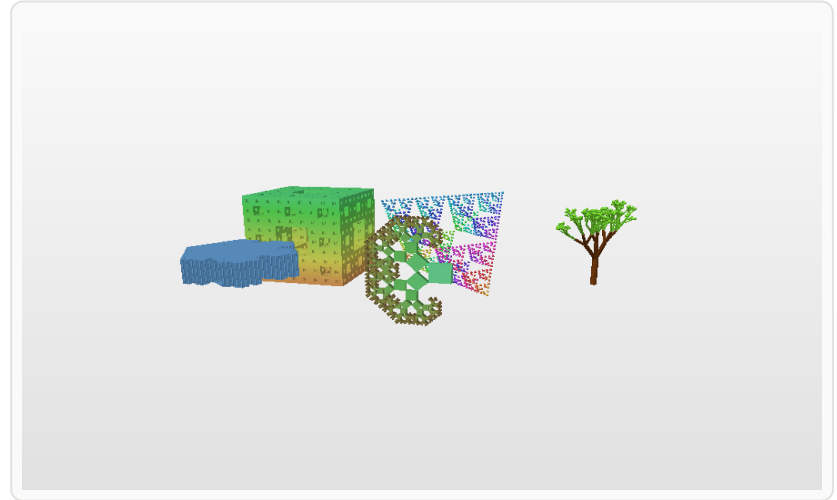
Įkėlimas atgal

load("raide.off") grąžina modelį, kurį galima stumdyti ir dažyti.

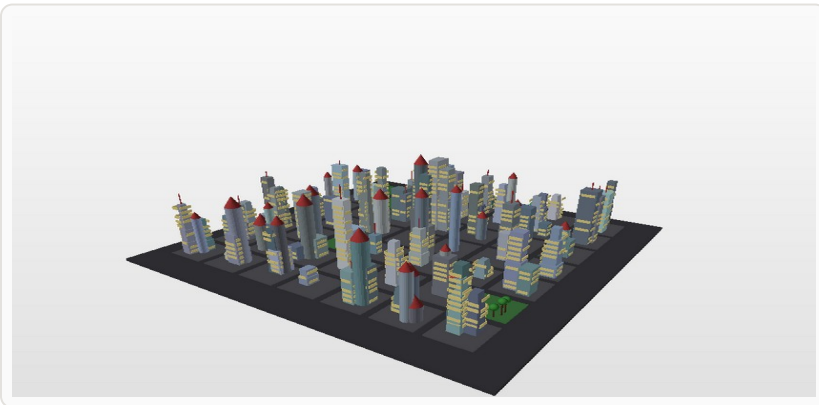
Didesni pavyzdžiai



18_chess_set.py – sukinys, sluoksniai, loginės operacijos



17_fractals.py – Mengerio kempinė, Sierpinskis, medis



Nuo 1.2 prie 2.0

add.py 1.2	add.py 2.0	kas tai
<code>cube2(c, e, b, RGB)</code>	frame	kubo briaunų karkasas
<code>cylinder2(A, B, r, k, RGB)</code>	tube	cilindras be dangtelių
<code>cylinder3(A, B, r, k, RGB)</code>	cup	uždarytas iš vieno galo
<code>cone2(A, B, r, k, RGB)</code>	cone_open	tik šoninis paviršius
<code>rectangle3D(c, e, RGB)</code>	cuboid	stačiakampis gretasienis
<code>newface(A, RGB)</code>	polygon	viena plokščia siena
<code>spin3D(A, B, S, ...)</code>	revolve	sukinys
<code>off(failas)</code>	save(failas)	.off, .obj, .ply arba .stl

Visi seni vardai tebeveikia. Dvylika 1.2 versijai rašytų modelių guli tests/legacy/ aplanke, ir testai tikrina, kad jie duotų lygiai tiek pat sienų kaip anksčiau.

Kur viską rasti

Modulis, pavyzdžiai, testai ir dokumentacija

github.com/.../add.py

Dokumentacija lietuviškai ir angliškai

...github.io/add.py

Klausimai

akataxis@gmail.com · martynas.sabaliauskas@mif.vu.lt

Ačiū už dėmesį.

